NTNU

Norwegian University of Science and Technology

Camera Manager

Technical Documentation 2026

2026 version made by Sewan Hamida, based on the 2024 and 2019 versions made by students, and on the 2014 version made by Thomas Dubrulle and Antonin Durey with Tomas Holt, Grethe Sandstrak, Alexander Holt and J.H. Nilsen. This document is intended for future developers who need to understand the 2026 modernization work, bug fixes, and safe extension points in the CameraManager project.

Document Map

1. Purpose of This Document

This document explains the technical work performed on CameraManager during the 2026 internship period. It is intended for a future student or developer who needs to understand the current state of the project, continue development, fix bugs, or adapt the software to another platform.

The document does not replace the user guide. The user guide explains how to install and operate the application. This document explains how the application is built internally, which parts were changed, why these changes were necessary, and where a developer should look when modifying the project.

The main goals of the 2026 work were:

- modernize the build system;
- make the project work with Qt 6;
- restore and stabilize camera settings;
- improve camera preview performance;
- restore usable TrackPoint preview performance after the Qt 6 migration;
- improve camera identification in the interface;
- add practical display features such as image rotation;
- make the project easier to build on macOS, Windows, and Linux.

---

2. Short Technical Summary

CameraManager is a Qt desktop application used to manage several industrial cameras, display live previews, configure camera parameters, and support TrackPoint/motion-capture workflows.

The application is built around three main layers:

1. The user interface layer, mainly `MainWindow`, menus, widgets, and OpenGL display widgets.
2. The camera abstraction layer, mainly `AbstractCamera`, `AbstractCameraManager`, and Spinnaker-specific subclasses.
3. The image-processing and visualization layer, mainly `ImageOpenGLWidget`, `Texture`, `ImageDetect`, TrackPoint data, and calibration/project viewers.

The most important technical changes are:

- the project now builds with CMake instead of qmake;
- Qt 5 compatibility leftovers were cleaned up or replaced for Qt 6;
- Spinnaker integration was made safer and more portable;
- camera properties are now read from and written to the real camera nodes correctly;

- camera nicknames and serial numbers are displayed in the interface;
- live camera widgets can be clicked to select the matching camera in the settings panel;
- black-and-white / color display switching was repaired;
- TrackPoint preview processing was moved away from the critical rendering path;
- display rotation was added in 90 degree steps;
- repeated noisy log messages such as `not streaming` were reduced.

---

3. Repository Reference

The current project repository is available on GitHub: Seeewan/CameraManager.

The Git history should be used as a general reference, but it should not be read as a complete chronological record of the work. There are relatively few commits because a significant part of the modernization and debugging work was done before the base repository was fully recovered and organized in Git.

For this reason, this technical document describes the final state of the code and the main categories of changes rather than listing each commit individually.

4. Build System Migration: qmake to CMake

Previous Situation

The original project used qmake project files such as:

- `camera_manager.pro`
- `camera_manager.pri`
- Visual Studio solution files such as `camera_manager.sln` and `CameraManager.sln`
- generated Qt resource output such as `qrc_cameramanager.cpp`

This made the build harder to maintain across platforms. qmake also made it less convenient to manage Qt 6, Spinnaker libraries, compiler definitions, runtime resources, and platform-specific library paths in one place.

Current Situation

The project now uses a main CMake file:

- `CMakeLists.txt`

The CMake configuration is responsible for:

- requiring C++17;
- finding Qt 6 Widgets and OpenGL modules;
- compiling all application source files;
- configuring the Spinnaker SDK location;
- linking Spinnaker libraries;
- copying runtime resources next to the executable;
- handling platform-specific differences between macOS, Windows, and Linux.

Why This Matters

CMake makes the project more standard for modern C++/Qt development. It is easier to open in Qt Creator, easier to configure on different operating systems, and easier to document for future users.

Where to Modify

Modify `CMakeLists.txt` when:

- adding a new `.cpp` or `.h` file;
- adding a new Qt module;
- changing the Spinnaker SDK path logic;
- adding platform-specific libraries;
- changing where runtime resources are copied.

When adding new source files, make sure they are included in the CMake target. If the file is not listed or not picked up by the build configuration, Qt Creator may show it in the project tree but it will not necessarily be compiled.

---

5. Qt 6 Migration and Cleanup

Previous Situation

The historical codebase contained Qt 5-era patterns and generated files. Some parts were compatible with newer Qt versions, while other parts needed updates. This created a mixed Qt 5 / Qt 6 situation where the application could compile in some environments but fail or behave incorrectly in others.

Examples of migration-sensitive areas include:

- Qt OpenGL includes;
- keyboard shortcut definitions;
- generated UI or resource files;

- build-system assumptions;
- platform-specific OpenGL/GLU includes.

Current Situation

The project has been cleaned up for Qt 6. The current code uses Qt 6 through CMake, and old qmake-related files were removed from the active build path.

Relevant files include:

- `CMakeLists.txt`
- `src/main.cpp`
- `src/menubar.cpp`
- `src/3DProject/displaywindow.h`
- `src/widgetgl.h`
- `src/texture.h`
- `src/qopenglvideowidget.h`
- `src/ui_mainwindow.h`

Notable Fixes

Keyboard shortcuts were updated to Qt 6-compatible syntax, for example using combinations such as `Qt::CTRL | Qt::Key_*`.

OpenGL-related includes were adjusted so that the application can compile on modern Qt 6 environments and across platforms.

Generated build artifacts are no longer treated as source files. In a CMake/Qt 6 project, generated files should normally be produced by the build system, not manually tracked or edited.

---

6. Runtime Resource Handling

Problem

CameraManager depends on runtime files, especially configuration files under the `CameraManager/` folder. If the application is started from a different working directory, relative paths can fail. This is common when launching from Qt Creator, Finder, Explorer, a terminal, or a packaged app.

Fix

The application now sets up a runtime root at startup. The code in `src/main.cpp` resolves where the executable is located and sets the current directory so that runtime files can be found consistently.

CMake also copies the `CameraManager/` runtime folder next to the built executable.

Where to Modify

Modify `src/main.cpp` if the runtime folder strategy changes.

Modify `CMakeLists.txt` if new runtime folders or files need to be copied after build.

Modify files under `CameraManager/props/` when changing default camera parameters, quick presets, or stored settings.

---

7. Spinnaker SDK Integration

Role of Spinnaker

Spinnaker is the SDK used to communicate with FLIR/Teledyne cameras such as the Blackfly S series. It provides access to:

- connected cameras;
- image acquisition;
- camera serial numbers and model names;
- GenICam nodes;
- exposure, gain, gamma, trigger, and acquisition settings;
- pixel format conversion.

Important Files

- `src/spincameramanager.cpp`
- `src/spincameramanager.h`
- `src/spincamera.cpp`
- `src/spincamera.h`
- `src/spincameraproperty.h`
- `src/spinnaker_compat.h`
- `src/systemmanager.cpp`
- `src/systemmanager.h`

Header Compatibility

A compatibility header was added:

- `src/spinnaker_compat.h`

This file helps the project support different Spinnaker installation layouts. Depending on the platform, headers may come from a system installation or from bundled/include paths configured by CMake.

Where to Modify

Modify Spinnaker-specific camera behavior in `src/spincamera.cpp` and `src/spincamera.h`.

Modify camera discovery and manager-level Spinnaker behavior in `src/spincameramanager.cpp` and `src/spincameramanager.h`.

Modify SDK path or linking behavior in `CMakeLists.txt`.

---

8. Camera Settings Repair

Original Problem

The camera settings panel existed in the interface, but according to testing, the settings did not actually work correctly. Changing values in the UI did not reliably update the real camera settings, and reading current values from the camera was incomplete or unsafe.

This is a common issue with industrial camera SDKs because camera parameters are exposed as GenICam nodes. Each node can have different access rules:

- readable or not readable;
- writable or not writable;
- integer, float, boolean, or enumeration;
- automatic mode available or unavailable;
- minimum and maximum values depending on camera model and current mode.

Writing directly without checking node availability can fail silently, throw SDK exceptions, or leave the UI out of sync.

Current Fix

The settings logic was rewritten around safer Spinnaker/GenICam access helpers in `src/spincamera.cpp`.

Important helper responsibilities include:

- checking whether a node exists;
- checking whether a node is readable;
- checking whether a node is writable;
- reading float values and ranges;
- reading and writing boolean values;
- reading and writing enumeration values;
- refreshing metadata before displaying values in the UI.

The settings are represented through `SpinCameraProperty` objects, and the UI is updated through `AbstractCameraManager`.

Important functions include:

- `SpinCamera::setSpinProperty(...)`
- `SpinCamera::refreshSpinPropertyMetadata(...)`
- `AbstractCameraManager::updateSpinProperties(...)`
- property save/load functions in `AbstractCameraManager`

Why It Works Better Now

The UI no longer assumes that every setting is always writable. It reads the camera capabilities and updates the controls accordingly. This reduces mismatches between the interface and the physical camera.

`QSignalBlocker` is used when refreshing UI controls. This is important because updating a spinbox, checkbox, or combo box from code can emit signals. Without blocking signals during refresh, the UI can accidentally write values back to the camera while it is only trying to display them.

Where to Modify

Modify `src/spincamera.cpp` when adding support for a new Spinnaker camera parameter.

Modify `src/spincameraproperty.h` if the structure used to describe a camera property needs new fields.

Modify `src/abstractcameramanager.cpp` if the way properties are displayed, saved, or loaded changes.

Modify `CameraManager/props/*.ini` if default or preset values need to change.

---

9. Camera Names and Nicknames

Problem

When several cameras are connected, identifying them only by index is not enough. For a motion-capture setup, the user needs to know which physical camera corresponds to which preview and which settings panel.

Fix

Camera naming support was added through the camera abstraction layer.

The camera object now stores or exposes:

- a default camera name;
- a custom nickname;
- a serial number;
- a model name.

The UI can display user-friendly names while still keeping the camera index and serial number available.

Important Files

- `src/abstractcamera.h`
- `src/abstractcamera.cpp`
- `src/abstractcameramanager.h`
- `src/abstractcameramanager.cpp`
- `src/spincamera.cpp`
- `src/mainwindow.cpp`

Typical Display Logic

The displayed title follows the idea:

```
Camera 0 - serial_number
```

If a custom nickname is available, the interface can use it to make the camera easier to identify.

Where to Modify

Modify `AbstractCameraManager::cameraDisplayTitle(...)` if the display format should change.

Modify `AbstractCamera` if more identity fields need to be stored.

Modify `SpinCamera` if the serial number or model name should be read differently from Spinnaker.

10. Serial Number Display

Problem

The camera preview windows previously did not clearly show the physical camera serial number. This made it hard to match a preview to a real camera, especially when several identical Blackfly cameras were connected.

Fix

The serial number is read from the Spinnaker camera information and displayed above the corresponding camera preview.

This gives titles such as:

`Camera 0 - 12345678`

Why It Matters

For a multi-camera motion-capture setup, serial numbers are more reliable than USB order or camera index. The operating system or SDK may enumerate cameras differently after reconnecting them. A serial number is tied to the physical device.

Where to Modify

Modify serial reading in `src/spincamera.cpp`.

Modify preview title formatting in `src/abstractcameramanager.cpp`.

Modify UI placement in `src/mainwindow.cpp` or the relevant camera display widget if the title layout changes.

11. Selecting Settings by Clicking the Camera Preview

Problem

The settings panel on the left could be changed by selecting a camera in the menu/list, but clicking directly on a camera preview did not select the corresponding camera settings. This was confusing because the live preview is the most visible representation of the camera.

Fix

The camera preview widget now exposes a click handler. When the user clicks a live camera display, `MainWindow` selects the matching camera in the left-side settings panel.

Important Files

- `src/imageopenglwidget.h`
- `src/imageopenglwidget.cpp`
- `src/mainwindow.h`
- `src/mainwindow.cpp`
- `src/abstractcameramanager.cpp`

Why It Matters

This connects the visual workflow and the configuration workflow. The user can now click the image they want to configure and immediately see the corresponding settings.

Where to Modify

Modify `ImageOpenGLWidget` if the click behavior should change.

Modify `MainWindow::selectCameraFromLiveView(...)` or equivalent selection logic if the target UI selection changes.

---

12. Black-and-White / Color Mode Fix

Original Problem

The switch between black-and-white and color mode did not work correctly. The interface had the option, but the camera acquisition/display pipeline did not handle the mode transition reliably.

Possible symptoms included:

- image not updating correctly after switching mode;
- wrong image container used;
- TrackPoint/project display still using assumptions from monochrome mode;
- preview windows not being reset correctly;
- incompatible processing while color mode was active.

Current Fix

The camera abstraction and Spinnaker acquisition path were updated to support both monochrome and color image containers.

Important changes include:

- `AbstractCamera` supports mono and color image storage;
- `SpinCamera::captureImage()` handles `PixelFormat_Mono8` and `PixelFormat_RGB8`;
- active camera display entries can reference the appropriate image window;
- color mode toggling resets live views and disables incompatible features when needed.

Important Files

- `src/abstractcamera.h`
- `src/abstractcamera.cpp`
- `src/spincamera.cpp`
- `src/abstractcameramanager.cpp`
- `src/mainwindow.cpp`

Why TrackPoint Is Affected

TrackPoint detection is based on image processing assumptions that are easier and faster on monochrome images. If the app switches to color, TrackPoint-related tools may need to be disabled or adapted to avoid incorrect results or unnecessary processing cost.

Where to Modify

Modify `SpinCamera::captureImage()` if a new pixel format is needed.

Modify `MainWindow::on_actionColor_toggled(...)` if the behavior of color mode changes.

Modify image processing functions if TrackPoint should eventually support color images directly.

---

13. Live Preview Performance and FPS

Problem

The camera preview could lag even when the UI displayed a high theoretical frame rate. In practice, there are two different frame rates:

- the camera acquisition frame rate;

- the display/rendering frame rate.

A camera can acquire at a high FPS while the UI renders slowly if image processing or UI updates block the main thread.

Current Improvements

The live preview pipeline was cleaned so that display updates are less blocked by extra processing. The camera image display is handled through OpenGL widgets and texture updates, while expensive operations are avoided or delayed when possible.

Important Files

- `src/videoopenglwidget.cpp`
- `src/videoopenglwidget.h`
- `src/imageopenglwidget.cpp`
- `src/imageopenglwidget.h`
- `src/texture.h`
- `src/mainwindow.cpp`

Key Principle

The UI thread should stay focused on displaying the most recent image. Expensive processing should not happen synchronously every time a frame is rendered.

Where to Modify

Modify `ImageOpenGLWidget` when changing how camera images are rendered.

Modify `Texture` if the OpenGL texture upload strategy changes.

Modify timer intervals in `MainWindow` if the global refresh behavior changes.

---

14. TrackPoint Preview Performance Fix

Original Problem

After the Qt 6 migration, enabling TrackPoint preview caused a large drop in perceived frame rate, especially when three cameras were connected. The interface could still display a high FPS value, but the actual visual update became slow.

The problem was worse with several cameras because TrackPoint preview processing was repeated for each stream and could block rendering.

Cause

The expensive TrackPoint detection work was too closely tied to the display/render path. If the application tries to detect TrackPoints synchronously for every displayed frame, the UI thread becomes overloaded.

For three cameras, the cost is multiplied. Even if each individual detection is acceptable, the total work can exceed what the CPU can process in real time.

Fix

TrackPoint processing was moved away from the immediate rendering path. `ImageOpenGLWidget` now uses asynchronous processing and cached TrackPoint results.

The current strategy is:

- keep rendering the latest camera image;
- process TrackPoint preview at a controlled interval;
- avoid launching unlimited processing jobs;
- reuse the last available TrackPoint results until new ones are ready;
- draw the overlay from cached results.

Important implementation concepts include:

- `TrackPointProcessingResult`;
- asynchronous processing with `std::async`;
- cached detected points;
- a preview interval such as `TRACKPOINT_PREVIEW_INTERVAL_MS`;
- conversion between image coordinates and display coordinates.

Important Files

- `src/imageopenglwidget.cpp`
- `src/imageopenglwidget.h`
- `src/imagedetect.h`
- `src/trackpointproperty.h`

Why This Improved Performance

Rendering can continue even while TrackPoint processing is running. The UI no longer waits for every detection pass before drawing the next frame. This reduces visible lag and makes the preview usable with multiple cameras.

Limits

This optimization improves preview responsiveness, but it does not make TrackPoint detection free. If the detection algorithm is CPU-heavy and three high-resolution cameras are processed continuously, CPU usage can still be high.

For future improvement, the project could move TrackPoint detection to a persistent worker thread pool, use GPU processing, reduce image resolution for preview detection, or process only regions of interest.

---

15. TrackPoint Memory Behavior

Observed Issue

During testing, RAM usage increased progressively when TrackPoint preview was enabled for a long time. This suggests that some temporary objects, cached images, futures, SDK images, or Qt/OpenGL resources may not be released quickly enough.

Current Understanding

The preview feature is designed for short-term visual checking, not necessarily for hours of continuous processing. However, RAM usage should ideally stabilize. A progressive memory increase is not ideal, even if short lab sessions remain usable.

Possible causes include:

- asynchronous jobs retaining image data longer than expected;
- repeated allocations in detection code;
- Spinnaker image buffers not being released quickly enough;
- Qt image conversions creating copies;
- cached TrackPoint results or display objects growing indirectly;
- OpenGL texture resources being recreated instead of reused.

Future Improvement

A future developer should profile memory while TrackPoint preview is active. Useful tools include:

- Activity Monitor on macOS;
- Instruments on macOS;
- Visual Studio Diagnostic Tools on Windows;
- Valgrind Massif or heaptrack on Linux;
- Qt Creator Analyzer tools if available.

The most likely files to inspect are:

- `src/imageopenglwidget.cpp`
- `src/imagedetect.h`
- `src/spincamera.cpp`
- `src/texture.h`

---

16. SIMD and Architecture Compatibility

Problem

Some image-detection code used SSE-related headers such as `<emmintrin.h>`. SSE is available on x86/x64 processors, but not on ARM processors such as Apple Silicon or ARM Linux systems.

This can break compilation on platforms like:

- Apple Silicon Macs;
- ARM Linux virtual machines;
- Raspberry Pi;
- other ARM devices.

Fix

The SIMD include logic was made conditional. The project only includes SSE headers when SIMD testing is enabled and the architecture supports it.

Important File

- `src/imagedetect.h`

Where to Modify

Modify `src/imagedetect.h` if adding architecture-specific optimizations.

A safe future direction would be to provide separate implementations for:

- generic CPU fallback;
- SSE x86/x64 path;
- NEON ARM path;
- optional GPU path.

---

17. Image Rotation Feature

Need

Some cameras may be physically mounted in different orientations. Instead of forcing the user to rotate the camera hardware or mentally compensate, the application now allows rotating a displayed camera image in quarter turns.

Implementation

A rotation button was added near the camera display controls. Clicking it rotates the displayed image by 90 degrees.

The implementation rotates the display mapping rather than modifying the original camera image data permanently.

Important concepts include:

- rotation state stored in the image widget;
- texture coordinate rotation;
- rotated image size calculation;
- coordinate conversion between displayed image coordinates and original image coordinates;
- TrackPoint overlay coordinates adjusted for rotation.

Important Files

- `src/imageopenglwidget.h`
- `src/imageopenglwidget.cpp`
- `src/mainwindow.cpp`

Why This Design Is Good

Rotating the display is less destructive than rotating the source image. The original image remains unchanged for acquisition, processing, and possible export. Only the user-facing preview is rotated.

Where to Modify

Modify `ImageOpenGLWidget::rotateClockwise(...)` or the rotation-related coordinate helpers if rotation behavior changes.

If future exported videos/images should also be rotated, that should be implemented separately in the recording/export pipeline rather than only in the preview widget.

---

18. Trigger Handling

What Trigger Means

A trigger is a signal that tells the camera when to acquire an image. Instead of the camera acquiring frames freely, an external device or software event can control capture timing.

There are two common trigger modes:

- hardware trigger: an electrical signal on a camera input line;
- software trigger: a command sent through the SDK.

For motion capture, hardware trigger can be important because several cameras need to capture at the same moment.

Current Application Behavior

CameraManager contains trigger-related configuration through Spinnaker. The code can configure trigger mode, trigger source, and acquisition mode. During testing, logs showed messages such as:

- trigger mode disabled;
- trigger source set to hardware;
- acquisition mode set to continuous;
- capturing value set to 1.

These logs describe how the camera acquisition state is being configured. They do not necessarily mean the external trigger wiring is working.

Important Files

- `src/spincamera.cpp`
- `src/spincamera.h`
- `src/mainwindow.cpp`

Developer Notes

If trigger behavior needs to be redesigned, the developer should first decide the intended final workflow:

- continuous preview only;
- software-triggered capture;
- hardware-triggered synchronized capture;
- recording video files;
- exporting TrackPoint positions;
- controlling cameras from another external application.

The UI should then be renamed or simplified to match that workflow. If the final use case is mostly external hardware triggering, the interface should make that clear.

---

19. Console Logs and `not streaming`

Problem

The application repeatedly printed `not streaming` in the logs. This was noisy and confusing because it looked like a serious error even when the camera was simply not currently acquiring.

Meaning

`not streaming` means the application tried to capture or update an image while the camera stream was not active. It does not mean internet streaming or video broadcasting. It refers to the camera acquisition stream between the physical camera and the software.

Fix

The message is now limited so it appears once until streaming resumes, instead of being printed continuously in a loop.

Important Files

- `src/spincamera.cpp`
- `src/spincamera.h`

Where to Modify

Modify the warning flag logic in `SpinCamera` if logging behavior changes.

A future improvement would be to replace raw console logs with a proper logging system with levels such as debug, info, warning, and error.

---

20. File and Folder Guide

This section explains the main files and what a future developer can safely modify.

Build and Runtime Files

`CMakeLists.txt`

Main build configuration. It defines the executable, Qt dependencies, C++ standard, Spinnaker paths, platform-specific settings, and resource copying.

Modify this file when adding source files, changing dependencies, or adapting the build to another platform.

`CameraManager/`

Runtime data folder copied next to the executable. It contains configuration files and resources needed while the application runs.

Do not assume this folder is only documentation. It is part of the runtime behavior.

`CameraManager/props/`

Stores camera settings presets and default property files.

Useful files include:

- `defaultCameraSettings.ini`
- `quickfile_camerasettings.ini`
- `spinCameraSettings.ini`

Modify these when changing default camera settings or saved preset behavior.

`src/main.cpp`

Application entry point. It initializes Qt and configures the runtime working directory.

Modify this file only if startup behavior or runtime path handling changes.

`src/constants.h`

Project-wide constants. Use this for shared values that should not be duplicated across the codebase.

Main Window and Interface

`src/mainwindow.h` and `src/mainwindow.cpp`

Central UI controller. It connects menus, camera managers, timers, preview widgets, settings panels, color mode, trigger actions, and TrackPoint preview behavior.

Modify this file when changing application-level workflow or connecting UI actions to backend behavior.

`src/ui_mainwindow.h`

Generated or maintained UI structure for the main window. Depending on the current workflow, this may be generated from a `.ui` file or manually tracked from an older setup.

Be careful when editing this file directly. If a `.ui` source exists or is reintroduced, generated files should not be manually modified.

`src/menubar.h` and `src/menubar.cpp`

Menu bar logic and shortcuts. This was touched during Qt 6 cleanup.

Modify this when adding or changing menu actions.

Camera Abstraction Layer

`src/abstractcamera.h` and `src/abstractcamera.cpp`

Base camera representation independent from Spinnaker. It stores common camera state such as image buffers, camera name, custom nickname, serial number, model, and common camera behavior.

Modify this if all camera backends need a new shared field or behavior.

`src/abstractcameramanager.h` and `src/abstractcameramanager.cpp`

Base manager for camera lists, active camera displays, camera settings UI, property save/load, display title generation, and camera selection.

Modify this when changing behavior shared by all camera managers.

`src/emptycameramanager.h` and `src/emptycameramanager.cpp`

Fallback or placeholder manager used when no real camera manager is available.

Useful for testing UI behavior without connected cameras.

Spinnaker Camera Backend

`src/spincameramanager.h` and `src/spincameramanager.cpp`

Spinnaker-specific camera manager. It discovers cameras through the Spinnaker system and creates `SpinCamera` objects.

Modify this when changing camera discovery, initialization, or shutdown behavior.

`src/spincamera.h` and `src/spincamera.cpp`

Main Spinnaker camera implementation. This file handles acquisition, trigger configuration, image conversion, serial/model reading, camera properties, streaming state, and SDK node access.

This is one of the most important files in the project.

Modify this when changing:

- exposure/gain/gamma behavior;
- acquisition mode;
- trigger configuration;
- pixel format handling;
- serial/model retrieval;
- Spinnaker error handling;
- stream start/stop behavior.

`src/spincameraproperty.h`

Data structure for Spinnaker camera properties displayed in the UI.

Modify this when adding metadata fields for camera settings.

`src/spinnaker_compat.h`

Compatibility include wrapper for Spinnaker headers.

Modify this if Spinnaker SDK include paths or versions change.

`src/systemmanager.h` and `src/systemmanager.cpp`

System-level management around camera backend setup. This helps organize camera manager creation and high-level application state.

Modify this when adding a new camera backend or changing how the app chooses which backend to use.

Image Display and OpenGL

`src/videoopenglwidget.h` and `src/videoopenglwidget.cpp`

OpenGL video display widget base. It supports rendering image data efficiently through OpenGL.

Modify this when changing low-level video rendering behavior.

`src/imageopenglwidget.h` and `src/imageopenglwidget.cpp`

Camera image display widget. This file now handles several important features:

- live preview display;
- click-to-select-camera behavior;
- TrackPoint preview overlay;
- asynchronous TrackPoint processing;
- image rotation;
- coordinate conversion between image and display.

This is the main file to inspect when debugging preview lag, rotation, or TrackPoint overlay alignment.

`src/texture.h`

OpenGL texture wrapper. It controls how image data is uploaded to the GPU for display.

Modify this only if changing texture format, upload logic, or OpenGL resource management.

`src/qopenglvideowidget.h`

Qt OpenGL video widget integration.

Modify this if the Qt OpenGL widget strategy changes.

TrackPoint and Image Detection

`src/imagedetect.h`

Image-processing utilities used for TrackPoint detection. It includes architecture-specific SIMD handling.

Modify this when changing the detection algorithm or adding CPU/GPU optimizations.

`src/trackpointproperty.h`

TrackPoint configuration structure.

Modify this when changing TrackPoint threshold, radius, display, or detection parameters.

Calibration, Projects, and Viewer Widgets

`src/configfilereader.h` and `src/configfilereader.cpp`

Reads project or configuration files.

Modify this when changing file formats.

`src/configfileviewerwidget.h` and `src/configfileviewerwidget.cpp`

Displays configuration files in the UI.

Modify this when changing how project configuration is shown to the user.

`src/calibrationfile.h` and `src/calibrationfile.cpp`

Represents calibration data.

Modify this if calibration file parsing or storage changes.

`src/calibrationviewerwidget.h` and `src/calibrationviewerwidget.cpp`

UI widget for viewing calibration information.

`src/calibrationfileopenglwidget.h` and `src/calibrationfileopenglwidget.cpp`

OpenGL-based calibration visualization.

`src/calibrationfilewidget.h` and `src/calibrationfilewidget.cpp`

Calibration file UI component.

Modify calibration-related files only after understanding the expected calibration file format.

Image and Socket Viewers

`src/imageviewerwidget.h` and `src/imageviewerwidget.cpp`

Widget for displaying images outside the live camera preview context.

`src/imagelabel.h` and `src/imagelabel.cpp`

Label-like image display helper.

`src/socketviewerwidget.h` and `src/socketviewerwidget.cpp`

Socket-related viewer. This may be useful if the application exchanges data with another tool.

Modify socket-related files if future work connects CameraManager to an external acquisition, trigger, or motion-capture application.

3D Project / Swapping Corrector

Files under `src/3DProject/` appear to support 3D visualization or correction workflows.

Examples include:

- `displaywindow.h`
- `displaywindow.cpp`
- other 3D project-related files in the folder

Modify this area only when working on 3D visualization, marker display, or swapping correction behavior.

---

21. How to Add a New Camera Setting

A future developer adding a new setting should follow this process:

4. Identify the exact Spinnaker/GenICam node name.
5. Check whether the node is readable and writable in SpinView.
6. Add or update the property representation in `SpinCameraProperty` if needed.
7. Read the node metadata in `SpinCamera::refreshSpinPropertyMetadata(...)`.
8. Write the value safely in `SpinCamera::setSpinProperty(...)`.
9. Update the UI in `AbstractCameraManager` if a new control is needed.

10. Test with a real camera connected.
11. Test what happens when the camera does not support the node.

Important rule: never assume all Spinnaker cameras expose the same nodes with the same access rules.

---

22. How to Add a New Camera Display Feature

For display features such as overlays, buttons, rotation, zoom, or selection:

12. Start in `ImageOpenGLWidget`.
13. Keep the original image data unchanged unless the feature truly needs to modify it.
14. Add display-only transformations when possible.
15. Update coordinate conversion helpers if overlays or mouse interactions are affected.
16. Check behavior with TrackPoint preview enabled.
17. Check behavior with multiple cameras.
18. Check behavior after switching color mode.

Features that only affect the preview should usually not be implemented in `SpinCamera`, because `SpinCamera` should represent acquisition, not UI display.

---

23. How to Work on TrackPoint Performance

When improving TrackPoint performance, avoid putting heavy processing directly in the paint/render function.

Recommended strategy:

- keep rendering fast;
- process detection asynchronously;
- limit processing frequency;
- reuse cached results;
- avoid launching multiple detection jobs for the same camera at the same time;
- avoid unnecessary image copies;
- profile before optimizing blindly.

Files to inspect:

- `src/imageopenglwidget.cpp`
- `src/imageopenglwidget.h`
- `src/imagedetect.h`

- `src/trackpointproperty.h`

Possible future optimizations:

- worker thread pool instead of `std::async`;
- lower-resolution preview processing;
- region-of-interest detection;
- GPU implementation;
- architecture-specific SIMD paths;
- memory pooling to reduce allocations.

---

24. How to Work on Cross-Platform Build Issues

If the project builds on one platform but not another, check in this order:

19. Qt version and kit selected in Qt Creator.
20. Spinnaker SDK installed for the correct CPU architecture.
21. `SPINNAKER_ROOT` or CMake cache paths.
22. Whether the Spinnaker library actually exists on disk.
23. Whether the architecture matches, for example x86_64 vs arm64/aarch64.
24. Whether OpenGL/GLU headers are available.
25. Whether old build cache files are still being reused.

Useful commands:

```
uname -m
file /path/to/libSpinnaker.so
cmake --fresh -S . -B build
```

On Linux, if Spinnaker is missing, the project may configure or link incorrectly. The Spinnaker SDK must be installed before building against real cameras.

---

25. Known Weak Points

TrackPoint Memory Growth

RAM usage can grow during TrackPoint preview. This should be profiled and stabilized if the preview is expected to run for long sessions.

Trigger UI Clarity

The current trigger controls may not perfectly match the final intended lab workflow. If another application is supposed to trigger acquisition externally, the UI should be redesigned around that use case.

Logging

The application still uses console-style logs in several places. A structured logger would make debugging easier.

Generated UI Files

Some UI files may still reflect historical generated-code workflows. Long term, it would be cleaner to clearly separate editable `.ui` files from generated headers.

Documentation of Data Output

The final expected output of the application should be clarified:

- live preview only;
- recorded images;
- recorded video;
- TrackPoint positions;
- synchronized frames from several cameras;
- communication with another acquisition program.

This affects future architecture decisions.

---

26. Testing Checklist After Modifications

After changing camera or display code, test the following:

- application starts without camera connected;
- application detects one camera;
- application detects three cameras;
- each preview displays the correct serial number;
- clicking a preview selects the correct camera settings;
- camera settings update the real camera;
- settings refresh does not overwrite values accidentally;

- black-and-white mode works;
- color mode works;
- switching between modes does not crash;
- preview stays responsive;
- TrackPoint preview works with one camera;
- TrackPoint preview works with three cameras;
- TrackPoint overlay is aligned after rotation;
- closing a camera preview does not crash;
- reconnecting cameras does not confuse serial numbers;
- build works on macOS;
- build works on Windows;
- build works on Linux with Spinnaker installed.

---

27. Suggested Future Development Roadmap

The next developer could improve the project in this order:

26. Clarify the final acquisition workflow with the lab: preview, recording, external trigger, TrackPoint output, or all of these.
27. Stabilize TrackPoint memory usage.
28. Replace ad-hoc logs with a structured logging system.
29. Simplify trigger controls in the UI.
30. Add a clear recording/export pipeline if the final application must produce videos or data files.
31. Add automated build documentation for macOS, Windows, and Linux.
32. Add small test modes that work without real cameras.
33. Add a developer README explaining common CMake options and Spinnaker paths.

---

28. Notes for a Future Student

The most important idea is to keep the responsibilities separated.

`SpinCamera` should communicate with the physical camera.

`AbstractCameraManager` should coordinate cameras and settings.

`MainWindow` should connect application-level UI actions.

`ImageOpenGLWidget` should display images and visual overlays.

`ImageDetect` should contain image-processing logic.

If a bug appears, first identify which layer owns the problem. For example:

- wrong exposure value: probably `SpinCamera` or settings UI;
- wrong camera selected: probably `AbstractCameraManager` or `MainWindow`;
- laggy preview: probably `ImageOpenGLWidget`, timers, or image processing;
- wrong serial number: probably `SpinCamera` information reading;
- build failure: probably `CMakeLists.txt`, Qt kit, or Spinnaker installation;
- rotated overlay mismatch: probably coordinate conversion in `ImageOpenGLWidget`.

This separation makes the project much easier to modify without creating new bugs.